

15.10.2024г.

Въведение 2.0

Python
Лекция 3

```
spam = {}
```

```
type(spam) → class 'dict'
```

```
my_list = ['spam'] * 100 → ['spam', 'spam', 'spam', ...]
```

100 пъти

print функция

- желателно е да използваме f-strings:

```
main_character = "Captain Jack Sparrow"
print(f"This is the tale of {main_character}")
```

This is the tale of Captain Jack Sparrow

```
heights = {"Mount Everest": 8849, "Musala": 2925}
```

```
for name, height in heights.items():
```

```
    print(f"{name:15} ==> {height:10}")
```

"Mount Everest ==> 8849"

"Musala ==> 2925"

"

- extend - може да добавя един лист в друг лист, но без втори []
- reverse - обръща елементите на места
- mutable обектите се подават "по референция"

Контролни структури

• if - elif - else

```
if a == 5:
```

```
    print("a is five")
```

```
elif a == 3 and not b == 2:
```

```
    print("a is three, but b is not 2")
```

```
else:
```

```
    print("a is something else or b is 2")
```

// Използваме

- and
- or
- not

if (с булеви променливи)

```
a = True
```

```
if a:  
    print("a is True")
```

```
if not a:  
    print("a is not True")
```

- False, None, 0, 0.0, 0j
 - празен низ
 - празен tuple, list, dict, set
- } приемат се за лъжа - False
всичко останало е True

Можем да кастваме:

- `int('5') # 5`
- `b = 10`
`str(b) # '10'`
- `nums = (1, 2, 3)`
`nums = list(nums)`
`nums # [1, 2, 3]`
- `str(['baba', 2]) # "['baba', 2]"`
- `int('gecet') # грешка ValueError`
- `a = "yes"`
`bool(a) # True`

if (тестове за принадлежност)

```
my-list = [1, 2, 3, 4]
```

```
if 1 in my-list:
```

```
    print('1 is in my-list')
```

```
if 5 not in my-list:
```

```
    print('5 is not in my list')
```


a = 10

while a > 5:

a -= 1

print(f"a is {a}")

a is 9

a is 8

a is 7

a is 6

a is 5

Всички колекции са итеруеми

- Някои могат да бъдат обхождани последователно поне веднъж
- други могат да бъдат обхождани многократно

for - обхожда структури от данни

• primes = [3, 5, 7, 9]

for e in primes:

print(e**2) # 9 25 49 81

• people = { 'bob': 25, 'john': 22, 'mit': 56 }

for name, age in people.items():

print(f"{name} is {age} years old".format(name, age))

bob is 25 years old

john is 22 years old

...

• for i in range(0, 20):

print(i)

0 1 2 ... 18 19

• for i in range(0, 20, 3):

print(i)

0 3 6 9 ... 15 18

номе и наобратно:

• for i in range(20, 0, -1):

print(i)

20 19 18 ... 2 1

• for i in range(20, 0, -3):

print(i)

20 17 14 ... 5 2

break и continue

- излизат от най-вътрешния цикъл

match / case

status = 400

match status:

case 400:

400

print("400")

case 401 | 403:

→ | ≡ OR

print("401 или 403")

case _:

→ default case

print("default")

Функции

- приема аргументи
- може да има return, в противен случай връща None
- не се описват типовете

Позиционни параметри

def multiply(a, b):

return a * b

→ позиционни

multiply(5, 10) # 50

multiply(5) → грешка

• колкото са аргументите, толкова параметъра после трябва да пореден на функцията

Именувани параметри:

def multiply(a, b=2):

return a * b

multiply(5) # 10

multiply(5, 10) # 50

• взимат стойността по подразбиране
• могат да се поредат в какъвто и да е ред

менш в брой аргументи

```
def varfunc (some, *args, *kwargs):
```

...

```
varfunc('hello', 1, 2, 3, name='Bob', age=12)
```

```
# some = 'hello'  
# args = (1, 2, 3)  
# kwargs = {'name': 'Bob',  
            'age': 12}
```

- позиционните → args, tuple от аргументи
- именуваните → kwargs, dict от имена и стойности

Редът е важен!

Range - връща итерируемо за интервал от числа

```
numbers = range(3)
```

```
# [0, 1, 2, 3] ?
```

```
numbers = range(10, 13)
```

```
# [10, 11, 12]
```

`map(function, iterable)` - създава колекция от резултатите от прилагането на func върху всеки ел. от iter.

`filter(func, iterable)` - създава колекция от елементите, които върнат True

`reduce(func, iterable)` - вика func с елементите на колекцията, докато сведе всичко до една стойност

`all(iterable)` - всички елементи се оценяват на True

`any(iterable)` - поне един от елементите се оценява с True

`lambda` функция - анонимна функция, която използва друга функция (пример: map или filter)